business-workflow-assistant.py

Business Workflow Assistant — Retail Analytics Dashboard (sample) · Colaberry AI-Project repository · saved as PDF on 2026-08-12

business-workflow-assistant.py

```
#!/usr/bin/env python3
"""
============================================================================
 BUSINESS WORKFLOW ASSISTANT — Retail Analytics Dashboard (sample)
============================================================================

WHAT THIS PROGRAM DOES, IN PLAIN ENGLISH
------------------------------------------------------------------------
Think of this program as a helpful analyst you can chat with. It:

  1. LOGS IN to Claude (Anthropic's AI) using a secret "API key" — this is
     like a password that lets this program talk to Claude on your behalf.

  2. LOADS a small SAMPLE retail sales dataset (already built into this
     file, so there's nothing to download or connect to). Pretend it's
     the sales data for a mid-size retail chain.

  3. LETS YOU HAVE A CONVERSATION with Claude about that data — ask
     questions like "What sold best last month?" or "Which region is
     struggling?" and Claude will answer using the sample numbers. You
     can ask several questions in a row and Claude will remember what
     you already talked about (that's what "multi-turn conversation"
     means).

  4. BUILDS A DASHBOARD REPORT on demand. When you type the command
     "report", Claude turns everything it knows into a neatly organized,
     computer-readable report (a JSON file) — the kind of structured
     data a real dashboard (like a chart on a website) could read and
     display. This is called "structured output": instead of Claude
     replying with a paragraph, it replies with data in an exact,
     predictable shape that a program can rely on.

  5. SAVES that report to a file next to this script, and also prints a
     friendly summary of it to your screen.

You do not need to know how to code to run this. Just follow the
"HOW TO RUN THIS" instructions below.

------------------------------------------------------------------------
HOW TO RUN THIS (for a non-technical person)
------------------------------------------------------------------------
STEP A — Install the one thing this program needs:
    Open a terminal (Command Prompt / PowerShell / Terminal) and run:

        pip install anthropic

STEP B — Get a Claude API key (your "password" to talk to Claude):
    1. Go to https://console.anthropic.com and sign in (or sign up).
    2. Create an API key under "API Keys".
    3. Copy the key — it looks like "sk-ant-...".

STEP C — Tell this program your API key. Two options:

    Option 1 (recommended — one time, per terminal session):
        Windows (PowerShell):   $env:ANTHROPIC_API_KEY = "sk-ant-your-key-here"
        Windows (cmd.exe):      set ANTHROPIC_API_KEY=sk-ant-your-key-here
        Mac/Linux:               export ANTHROPIC_API_KEY="sk-ant-your-key-here"

    Option 2: the program will ask you to paste it in if it can't find
    one — nothing is saved to disk, it's only used for this one run.

STEP D — Run the program:

        python business-workflow-assistant.py

Then just type your questions and press Enter. Type "report" any time
```

```
to generate the structured dashboard report. Type "quit" to exit.
============================================================================
"""

# ---------------------------------------------------------------------------
# STEP 1 OF THE CODE: Bring in the tools this program needs.
#   - "os" and "getpass" help us read your API key safely.
#   - "sys" lets us exit cleanly if something goes wrong.
#   - "json" and "pathlib" help us save the report to a file.
#   - "datetime" timestamps the report.
#   - "anthropic" is Anthropic's official toolkit for talking to Claude.
#   - "pydantic" describes the exact shape of the structured report we
#     want back from Claude (it is installed automatically alongside the
#     anthropic package, so no extra install step is needed).
# ---------------------------------------------------------------------------
import getpass
import json
import os
import sys
from datetime import datetime
from pathlib import Path
from typing import List, Optional

try:
    import anthropic
    from pydantic import BaseModel, Field
except ImportError:
    print(
        "\nThis program needs the 'anthropic' package, which isn't installed yet.\n"
        "Please open a terminal and run:\n\n"
        "    pip install anthropic\n\n"
        "Then run this program again.\n"
    )
    sys.exit(1)


# ---------------------------------------------------------------------------
# STEP 2 OF THE CODE: Pick which Claude model to use.
#   Claude Opus 5 is Anthropic's flagship model — strong reasoning, good
#   at following instructions precisely, which matters when we ask it to
#   return exactly-shaped structured data later on.
# ---------------------------------------------------------------------------
MODEL_NAME = "claude-opus-5"


# ---------------------------------------------------------------------------
# STEP 3 OF THE CODE: The SAMPLE retail dataset.
#   In a real company this would come from a database or a spreadsheet.
#   For this demo, it's just written directly into the program so you can
#   run everything immediately with no setup. Feel free to edit these
#   numbers to experiment.
# ---------------------------------------------------------------------------
SAMPLE_RETAIL_DATA = [
    {"product": "Wireless Earbuds", "category": "Electronics", "region": "West",
     "month": "2026-06", "units_sold": 1240, "revenue_usd": 74400},
    {"product": "Wireless Earbuds", "category": "Electronics", "region": "East",
     "month": "2026-06", "units_sold": 860, "revenue_usd": 51600},
    {"product": "Yoga Mat", "category": "Fitness", "region": "West",
     "month": "2026-06", "units_sold": 430, "revenue_usd": 12900},
    {"product": "Yoga Mat", "category": "Fitness", "region": "South",
     "month": "2026-06", "units_sold": 610, "revenue_usd": 18300},
    {"product": "Stainless Water Bottle", "category": "Home & Kitchen", "region": "East",
     "month": "2026-06", "units_sold": 980, "revenue_usd": 19600},
    {"product": "Stainless Water Bottle", "category": "Home & Kitchen", "region": "South",
     "month": "2026-06", "units_sold": 720, "revenue_usd": 14400},
    {"product": "Desk Lamp", "category": "Home & Kitchen", "region": "West",
     "month": "2026-06", "units_sold": 310, "revenue_usd": 9300},
    {"product": "Desk Lamp", "category": "Home & Kitchen", "region": "North",
     "month": "2026-06", "units_sold": 145, "revenue_usd": 4350},
    {"product": "Running Shoes", "category": "Fitness", "region": "North",
     "month": "2026-06", "units_sold": 275, "revenue_usd": 24750},
    {"product": "Running Shoes", "category": "Fitness", "region": "South",
     "month": "2026-06", "units_sold": 190, "revenue_usd": 17100},
    {"product": "Bluetooth Speaker", "category": "Electronics", "region": "North",
     "month": "2026-06", "units_sold": 95, "revenue_usd": 6650},
    {"product": "Bluetooth Speaker", "category": "Electronics", "region": "East",
```

```
     "month": "2026-06", "units_sold": 205, "revenue_usd": 14350},
]


# ---------------------------------------------------------------------------
# STEP 4 OF THE CODE: Define the exact SHAPE of the dashboard report.
#
#   This is the heart of "structured output". Instead of hoping Claude's
#   reply happens to look like what we want, we describe the exact fields
#   we require (a "schema"), and the API guarantees Claude's answer will
#   match it. That means our program can reliably read report.headline,
#   report.key_metrics, etc. without ever getting a surprise format.
# ---------------------------------------------------------------------------
class KeyMetric(BaseModel):
    name: str = Field(description="Short label for the metric, e.g. 'Total Revenue'")
    value: str = Field(description="The metric's value, formatted for display, e.g. '$267,750'")
    trend: str = Field(description="One of: up, down, flat — the direction this metric is heading")


class TopProduct(BaseModel):
    product_name: str
    units_sold: int
    revenue_usd: float


class RegionalPerformance(BaseModel):
    region: str
    revenue_usd: float
    standout_note: str = Field(description="One short sentence on what's notable about this region")


class RetailDashboardReport(BaseModel):
    report_title: str
    generated_at: str = Field(description="Human-readable date/time this report covers")
    headline_summary: str = Field(description="2-3 sentence plain-English executive summary")
    key_metrics: List[KeyMetric]
    top_products: List[TopProduct]
    regional_performance: List[RegionalPerformance]
    recommendations: List[str] = Field(description="Concrete, actionable next steps for the business")
    risk_flags: List[str] = Field(description="Anything concerning that leadership should know about")


# ---------------------------------------------------------------------------
# STEP 5 OF THE CODE: Authenticate to the Claude API.
#
#   "Authenticate" just means: prove to Anthropic's servers that we're
#   allowed to use their AI, using the API key as proof. We check for the
#   key in an environment variable first (the recommended, more secure
#   way), and if it's missing, we ask you to paste it in just for this
#   run. We then make one tiny, cheap test request to confirm the key
#   actually works before we do anything else — so if something's wrong,
#   you find out immediately with a clear message instead of a confusing
#   crash later.
# ---------------------------------------------------------------------------
def authenticate_to_claude() -> anthropic.Anthropic:
    api_key = os.environ.get("ANTHROPIC_API_KEY")

    if not api_key:
        print(
            "No ANTHROPIC_API_KEY environment variable was found.\n"
            "You can paste your key here just for this run (it will not be saved to disk)."
        )
        api_key = getpass.getpass("Paste your Anthropic API key: ").strip()

    if not api_key:
        print("\nNo API key was provided, so this program can't continue. Exiting.")
        sys.exit(1)

    client = anthropic.Anthropic(api_key=api_key)

    # A minimal, cheap request just to confirm the key works before we
    # start the real conversation.
    try:
        client.messages.create(
            model=MODEL_NAME,
            max_tokens=8,
```

```python
            messages=[{"role": "user", "content": "Reply with the single word: OK"}],
        )
    except anthropic.AuthenticationError:
        print(
            "\nThat API key was rejected by Anthropic's servers (authentication failed).\n"
            "Double-check you copied the full key from https://console.anthropic.com "
            "and try again."
        )
        sys.exit(1)
    except anthropic.PermissionDeniedError:
        print(
            "\nThis API key doesn't have permission to use the requested model.\n"
            "Check your Anthropic Console account settings and try again."
        )
        sys.exit(1)
    except anthropic.APIConnectionError:
        print(
            "\nCouldn't reach Anthropic's servers. Please check your internet connection "
            "and try again."
        )
        sys.exit(1)
    except anthropic.APIStatusError as e:
        print(f"\nAnthropic's servers returned an unexpected error ({e.status_code}). "
              "Please try again in a moment.")
        sys.exit(1)

    print("Connected to Claude successfully.\n")
    return client


# ---------------------------------------------------------------------------
# STEP 6 OF THE CODE: Build the "system prompt".
#
#   The system prompt is Claude's standing instructions — who it should
#   act as, and what data it has access to. We hand Claude the sample
#   retail dataset here, once, so it doesn't need to be repeated in every
#   question. We also mark this block as "cacheable" — a cost-saving
#   feature where Anthropic's servers remember this large, unchanging
#   block between requests instead of reprocessing it every single time
#   you ask a new question.
# ---------------------------------------------------------------------------
def build_system_prompt() -> list:
    dataset_as_text = json.dumps(SAMPLE_RETAIL_DATA, indent=2)

    instructions = (
        "You are a retail analytics assistant helping a business owner understand "
        "their sales performance. You have been given one month of sample sales data "
        "below, broken down by product, category, region, units sold, and revenue.\n\n"
        "When answering questions, use only the numbers in this dataset. Keep answers "
        "clear, brief, and free of jargon — the person you're talking to is not a "
        "data analyst.\n\n"
        f"SAMPLE SALES DATA (JSON):\n{dataset_as_text}"
    )

    return [
        {
            "type": "text",
            "text": instructions,
            # Caching this block means repeated questions in the same
            # conversation are faster and cheaper, because the (fairly
            # large) dataset text doesn't need to be re-processed from
            # scratch every time.
            "cache_control": {"type": "ephemeral"},
        }
    ]


# ---------------------------------------------------------------------------
# STEP 7 OF THE CODE: Manage a multi-turn conversation.
#
#   The Claude API itself doesn't "remember" anything between requests —
#   every request is independent. To make it feel like a real back-and-
#   forth conversation, this class keeps a running transcript (a list of
#   who-said-what) and sends the whole transcript back to Claude on every
#   turn. Claude then sees the full history and can refer back to things
#   you asked earlier.
```

```python
# ---------------------------------------------------------------------------
class ConversationManager:
    def __init__(self, client: anthropic.Anthropic, system_prompt: list):
        self.client = client
        self.system_prompt = system_prompt
        self.history: List[dict] = []

    def ask(self, user_message: str) -> str:
        self.history.append({"role": "user", "content": user_message})

        try:
            response = self.client.messages.create(
                model=MODEL_NAME,
                max_tokens=2048,
                system=self.system_prompt,
                messages=self.history,
                # Adaptive thinking lets Claude decide on its own whether a
                # question needs extra reasoning steps before answering.
                thinking={"type": "adaptive"},
                # "medium" effort keeps everyday chat questions fast and
                # inexpensive; we'll ask for more effort later, only when
                # generating the full structured report.
                output_config={"effort": "medium"},
            )
        except anthropic.RateLimitError:
            return ("Claude is temporarily rate-limited (too many requests too fast). "
                    "Please wait a few seconds and ask again.")
        except anthropic.APIConnectionError:
            return "Lost connection to Anthropic's servers. Please check your internet and try again."
        except anthropic.APIStatusError as e:
            return f"Anthropic's servers returned an error ({e.status_code}). Please try again."

        if response.stop_reason == "refusal":
            reply_text = ("Claude declined to answer that one. Try rephrasing the question.")
        else:
            reply_text = next(
                (block.text for block in response.content if block.type == "text"),
                "(Claude didn't return a text answer for that question.)",
            )

        self.history.append({"role": "assistant", "content": response.content})
        return reply_text


# ---------------------------------------------------------------------------
# STEP 8 OF THE CODE: Generate the structured dashboard report.
#
#   This is where "structured output" happens. We ask Claude the same
#   question every time ("build the dashboard report") but instead of
#   letting it reply in free-form prose, we pass output_format=RetailDashboardReport.
#   The API then guarantees the reply matches that exact schema, and
#   response.parsed_output gives us back a ready-to-use Python object —
#   no fragile text-parsing required.
# ---------------------------------------------------------------------------
def generate_structured_dashboard_report(
    client: anthropic.Anthropic, system_prompt: list, conversation: ConversationManager
) -> RetailDashboardReport:
    print("\nBuilding your structured dashboard report... this takes a few seconds.\n")

    request_text = (
        "Using the sample sales data you were given, and anything relevant from our "
        "conversation so far, build the full retail analytics dashboard report."
    )

    messages_for_report = conversation.history + [{"role": "user", "content": request_text}]

    response = client.messages.parse(
        model=MODEL_NAME,
        max_tokens=4096,
        system=system_prompt,
        messages=messages_for_report,
        output_format=RetailDashboardReport,
    )

    return response.parsed_output
```

```

# ---------------------------------------------------------------------------
# STEP 9 OF THE CODE: Save the report to a file and print a friendly summary.
# ---------------------------------------------------------------------------
def save_and_display_report(report: RetailDashboardReport) -> None:
    output_path = Path(__file__).resolve().parent / "retail_dashboard_report.json"
    output_path.write_text(report.model_dump_json(indent=2), encoding="utf-8")

    print("=" * 70)
    print(f" {report.report_title}")
    print(f" Generated: {report.generated_at}")
    print("=" * 70)
    print(f"\n{report.headline_summary}\n")

    print("KEY METRICS")
    for metric in report.key_metrics:
        arrow = {"up": "^", "down": "v", "flat": "="}.get(metric.trend, "?")
        print(f"  [{arrow}] {metric.name}: {metric.value}")

    print("\nTOP PRODUCTS")
    for product in report.top_products:
        print(f"  - {product.product_name}: {product.units_sold} units, "
              f"${product.revenue_usd:,.0f} revenue")

    print("\nREGIONAL PERFORMANCE")
    for region in report.regional_performance:
        print(f"  - {region.region}: ${region.revenue_usd:,.0f} — {region.standout_note}")

    print("\nRECOMMENDATIONS")
    for i, rec in enumerate(report.recommendations, 1):
        print(f"  {i}. {rec}")

    if report.risk_flags:
        print("\nRISK FLAGS")
        for flag in report.risk_flags:
            print(f"  ! {flag}")

    print(f"\nFull report saved to: {output_path}")
    print("=" * 70 + "\n")


# ---------------------------------------------------------------------------
# STEP 10 OF THE CODE: The main program loop.
#
#   This ties everything together: log in, greet the user, then repeatedly
#   read a line of input and either (a) generate the structured report,
#   (b) exit, or (c) treat it as a question and pass it to Claude.
# ---------------------------------------------------------------------------
def main() -> None:
    print(
        "\n"
        "============================================================\n"
        " Business Workflow Assistant — Retail Analytics Dashboard\n"
        "============================================================\n"
    )

    client = authenticate_to_claude()
    system_prompt = build_system_prompt()
    conversation = ConversationManager(client, system_prompt)

    print(
        "I have a sample month of retail sales data loaded and ready.\n"
        "Ask me anything about it — e.g. 'What was our best-selling product?'\n"
        "or 'Which region underperformed?'\n\n"
        "Commands:\n"
        "  report   -> build the full structured dashboard report (saved as JSON)\n"
        "  quit     -> exit the program\n"
    )

    while True:
        try:
            user_input = input("You: ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nGoodbye!")
            break

```

```
        if not user_input:
            continue

        if user_input.lower() in ("quit", "exit"):
            print("Goodbye!")
            break

        if user_input.lower() == "report":
            try:
                report = generate_structured_dashboard_report(client, system_prompt, conversation)
                save_and_display_report(report)
            except anthropic.APIStatusError as e:
                print(f"Couldn't build the report right now (server error {e.status_code}). "
                      "Please try again.")
            except anthropic.APIConnectionError:
                print("Couldn't build the report — lost connection. Please check your internet.")
            continue

        answer = conversation.ask(user_input)
        print(f"\nClaude: {answer}\n")


if __name__ == "__main__":
    main()
```

Source: business-workflow-assistant.py — C:\Users\Regina\Documents\AI-Project\business-workflow-assistant.py